

Manual Técnico

Perfil Programador y Administrador

Sistema Funerario

Servidor Dell PowerEdge T150 · Ubuntu Server · Docker · PostgreSQL · Django · Nginx · Tailscale

Generado: 2026-06-17 19:46:54

Manual Técnico para Programador y Administrador

Sistema Funerario Versión documentada: 2026-06-17 Servidor objetivo: Dell PowerEdge T150 con Ubuntu Server, Docker, PostgreSQL, Nginx y Tailscale

1. Objetivo del Manual

Este manual está diseñado para dos perfiles:

- Administrador del sistema: persona responsable de operar el servidor, revisar respaldos, crear usuarios, reiniciar servicios, restaurar datos y mantener el acceso local/remoto.
- Programador: persona responsable de entender, modificar, probar y desplegar cambios en el código fuente.

El sistema funerario permite administrar fichas de clientes, pagos, documentos, contrato imprimible y contrato PDF. Está pensado para funcionar en red local y también desde fuera de la red mediante Tailscale.

2. Resumen Ejecutivo

El sistema está construido con Django y se ejecuta en contenedores Docker.

Componentes principales:

```
Chrome / Navegador
  |
  v
Nginx :8080
  |
  v
Django + Gunicorn
  |
  +--> PostgreSQL: clientes, pagos, usuarios
  |
  +--> Media: documentos subidos
```

Rutas importantes en producción:

/opt/sistema-funerario	Código fuente de la aplicación
/srv/funeraria/data	Datos activos
/srv/funeraria/data/postgres	Base de datos PostgreSQL
/srv/funeraria/data/media	Documentos subidos
/mnt/funeraria-backups	Respaldos

3. Arquitectura de Producción

El despliegue usa docker-compose.yml con tres servicios.

Servicio	Función
db	Base de datos PostgreSQL 16
web	Django ejecutado con Gunicorn
nginx	Servidor frontal en puerto 8080

Diagrama de servicios:

```
Usuario
  |
  | http://192.168.0.22:8080
  | https://servidor-funeraria.taild7534a.ts.net/
  v
Nginx
  |
  | proxy_pass http://web:8000
  v
Gunicorn / Django
  |
  +-- PostgreSQL db:5432
  |
  +-- /app/media -> /srv/funeraria/data/media
```

3.1 Nginx

Archivo:

`deploy/nginx/default.conf`

Responsabilidades:

- publicar el sistema en el puerto 8080;
- servir archivos estáticos desde `/static/`;
- reenviar solicitudes a Django;
- limitar subida de archivos a 25m;
- conservar encabezados `Host`, `X-Real-IP` y `X-Forwarded-*`.

3.2 Django / Gunicorn

Archivos principales:

`funeraria/settings.py`
`funeraria/urls.py`
`clientes/`

Responsabilidades:

- autenticación;
- CRUD de clientes;
- registro de pagos;
- subida y descarga de documentos;
- generación de contrato imprimible;
- generación de contrato PDF;
- validación de RUT chileno;
- cálculo de totales, pagos y saldos.

3.3 PostgreSQL

PostgreSQL guarda:

- clientes;
- pagos;
- usuarios;
- referencias a documentos;
- permisos y sesiones de Django.

No guarda los archivos subidos como binario. Los documentos viven en disco.

3.4 Media / Documentos

Los archivos subidos quedan en:

`/srv/funeraria/data/media`

Ejemplo:

`/srv/funeraria/data/media/clientes/juan-perez-12345678_5/contrato.pdf`

4. Manual para Administrador

Esta sección explica la operación diaria y tareas de mantenimiento.

4.1 Entrar al Servidor

Desde Windows PowerShell:

```
ssh funerariaadmin@192.168.0.22
```

Si está fuera de la red y Tailscale está activo, puede usar la IP o nombre Tailscale.

4.2 Ver Estado del Sistema

```
cd /opt/sistema-funerario
sudo docker compose ps
```

Estado esperado:

```
db          Up healthy
web         Up
nginx       Up
```

4.3 Abrir el Sistema

En red local:

```
http://192.168.0.22:8080
```

Por Tailscale:

```
https://servidor-funeraria.taild7534a.ts.net/
```

4.4 Ver Logs

Aplicación Django:

```
sudo docker compose logs -f web
```

Nginx:

```
sudo docker compose logs -f nginx
```

PostgreSQL:

```
sudo docker compose logs -f db
```

Últimas 100 líneas:

```
sudo docker compose logs --tail=100 web
```

4.5 Reiniciar Servicios

Reiniciar todo:

```
cd /opt/sistema-funerario
sudo docker compose restart
```

Reiniciar solo la aplicación:

```
sudo docker compose restart web
```

Reiniciar solo Nginx:

```
sudo docker compose restart nginx
```

4.6 Apagar y Encender

Apagar:

```
cd /opt/sistema-funerario
sudo docker compose down
```

Encender:

```
sudo docker compose up -d
```

Esto no borra datos porque los datos están en /srv/funeraria/data.

4.7 Crear Usuario Administrador

```
cd /opt/sistema-funerario
sudo docker compose exec web python manage.py createsuperuser
```

4.8 Cambiar Contraseña de Usuario

```
sudo docker compose exec web python manage.py changepassword nombre_usuario
```

4.9 Panel Administrativo

URL:

```
http://192.168.0.22:8080/admin/
```

Permite administrar:

- usuarios;
- grupos;
- clientes;
- pagos;
- documentos.

Recomendación:

Crear un usuario por persona. No compartir la cuenta admin.

5. RespalDOS para Administrador

5.1 RespalDO Completo Manual

Este comando respalda base de datos, documentos y configuración:

```
cd /opt/sistema-funerario
sudo ./deploy/scripts/backup.sh
```

Ver respaldos:

```
ls -lah /mnt/funeraria-backups
```

Ver contenido:

```
ls -lah /mnt/funeraria-backups/AAAA-MM-DD_HH-MM-SS
```

Debe contener:

```
base-datos.dump
documentos.tar.gz
configuracion.env
SHA256SUMS
```

5.2 Verificar RespalDO

```
cd /mnt/funeraria-backups/AAAA-MM-DD_HH-MM-SS
sha256sum -c SHA256SUMS
```

Debe mostrar OK.

5.3 RespalDOS Automáticos

Ver timer:

```
sudo systemctl status funeraria-backup.timer
```

Ejecutar ahora:

```
sudo systemctl start funeraria-backup.service
```

Ver historial:

```
journalctl -u funeraria-backup.service --no-pager -n 80
```

5.4 Restaurar Respaldo Completo

Advertencia:

Restaurar reemplaza la base de datos actual por la del respaldo.

Primero, si el sistema todavía funciona, cree un respaldo de emergencia:

```
cd /opt/sistema-funerario
sudo ./deploy/scripts/backup.sh
```

Restaurar:

```
sudo ./deploy/scripts/restore.sh /mnt/funeraria-backups/AAAA-MM-DD_HH-MM-SS
```

Ejemplo:

```
sudo ./deploy/scripts/restore.sh /mnt/funeraria-backups/2026-06-14_18-30-00
```

Después pruebe:

- login;
- lista de clientes;
- abrir ficha;
- pagos;
- documentos;
- PDF.

6. Falla Crítica del Programa

Use este procedimiento si el código se dañó o una actualización rompió el sistema.

Datos que no debe borrar:

```
/srv/funeraria/data
/mnt/funeraria-backups
```

6.1 Respaldo de Emergencia

```
cd /opt/sistema-funerario
sudo ./deploy/scripts/backup.sh
```

Si no funciona, al menos copie .env:

```
cp /opt/sistema-funerario/.env /mnt/funeraria-backups/env_emergencia_$(date +%Y-%m-%d_%H-%M-%S)
```

6.2 Detener Sistema

```
cd /opt/sistema-funerario
sudo docker compose down
```

6.3 Guardar Código Roto

```
STAMP="$(date +%Y-%m-%d_%H-%M-%S)"
cd /opt
sudo mv sistema-funerario "sistema-funerario_roto_$STAMP"
```

6.4 Copiar Código Nuevo

Desde Windows:

```
scp "C:\proyectos\sistema-funerario-final.zip"
funerariaadmin@192.168.0.22:/tmp/
```

En el servidor:

```
cd /tmp
rm -rf sistema-funerario-nuevo
mkdir sistema-funerario-nuevo
unzip -q /tmp/sistema-funerario-final.zip -d /tmp/sistema-funerario-nuevo

sudo mkdir -p /opt/sistema-funerario
sudo cp -a /tmp/sistema-funerario-nuevo/sistema-funerario-final/. /opt/sistema-funerario/
```

6.5 Recuperar .env

```
sudo cp "/opt/sistema-funerario_roto_$STAMP/.env" /opt/sistema-funerario/.env
```

Si no existe:

```
sudo cp /mnt/funeraria-backups/AAAA-MM-DD_HH-MM-SS/configuracion.env /opt/sistema-funerario/.env
```

6.6 Permisos

```
sudo chown -R funerariaadmin:funerariaadmin /opt/sistema-funerario
cd /opt/sistema-funerario
sudo chmod +x deploy/scripts/*.sh
```

6.7 Reconstruir

```
cd /opt/sistema-funerario
sudo docker compose up -d --build
```

Verificar:

```
sudo docker compose ps
sudo docker compose logs --tail=100 web
```

7. Manual para Programador

Esta sección explica cómo entender y modificar el código.

7.1 Estructura del Proyecto

```
sistema-funerario-final/
├── clientes/
├── funeraria/
├── deploy/
├── static/
├── tests/
├── docs/
├── Dockerfile
├── docker-compose.yml
├── requirements.txt
└── manage.py
```

7.2 Aplicación clientes

Es el núcleo del sistema.

Archivos:

Archivo	Función
`models.py`	Modelos Client, Payment y ClientDocument
`forms.py`	Formularios de cliente, pago y documento
`views.py`	Lógica web de clientes, pagos, documentos y contratos
`urls.py`	Rutas de la app
`validators.py`	Normalización y validación de RUT
`admin.py`	Configuración de `/admin/`

```
| `pdf_fallback.py` | PDF alternativo con ReportLab |
| `templates/` | HTML |
| `static/clientes/app.css` | Estilos CSS |
```

7.3 Modelos Principales

Client

Representa la ficha del cliente.

Campos críticos:

```
name
rut
folio
service_net
vat_rate
document_folder
created_by
created_at
updated_at
```

Reglas:

- name y rut son obligatorios;
- rut es único;
- folio se genera automáticamente;
- document_folder se genera automáticamente;
- service_total, paid_total y balance se calculan como propiedades.

Payment

Representa un pago.

Campos:

```
client
payment_date
amount
receipt_number
notes
created_at
```

ClientDocument

Representa un archivo subido.

Campos:

```
client
category
file
original_name
uploaded_at
uploaded_by
```

7.4 Vistas Principales

```
| Vista | Función |
|---|---|
| `client_list` | Lista y búsqueda |
| `client_create` | Crear ficha |
| `client_update` | Editar ficha |
| `client_detail` | Ver ficha |
| `client_delete` | Eliminar ficha |
| `payment_create` | Registrar pago |
| `payment_delete` | Eliminar pago |
| `document_upload` | Subir documento |
| `document_download` | Descargar documento |
| `document_delete` | Eliminar documento |
| `contract_print` | Contrato imprimible |
| `contract_pdf` | Contrato PDF |
```

Todas las vistas relevantes usan login_required.

7.5 Rutas

Archivo:

clientes/urls.py

Rutas clave:

/	Lista
/clientes/nuevo/	Crear
/clientes/<id>/	Detalle
/clientes/<id>/editar/	Editar
/clientes/<id>/pagos/nuevo/	Nuevo pago
/clientes/<id>/contrato/	Imprimir
/clientes/<id>/contrato.pdf	PDF

7.6 Configuración Global

Archivo:

funeraria/settings.py

Puntos importantes:

- lee variables desde .env;
- usa PostgreSQL si existe POSTGRES_HOST;
- usa SQLite si no existe POSTGRES_HOST;
- define MEDIA_ROOT;
- define STATIC_ROOT;
- configura login y logout;
- configura seguridad de cookies.

7.7 Desarrollo Local

En Windows:

```
python -m venv .env
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python manage.py migrate
python manage.py createsuperuser
python manage.py runserver
```

Abrir:

http://127.0.0.1:8000

7.8 Desarrollo con Docker

```
cp .env.example .env
docker compose up -d --build
docker compose exec web python manage.py createsuperuser
```

Abrir:

http://127.0.0.1:8080

7.9 Migraciones

Cuando cambie modelos:

```
python manage.py makemigrations
python manage.py migrate
```

En producción:

```
sudo docker compose up -d --build
```

El script `entrypoint.sh` ejecuta migraciones automáticamente.

7.10 Pruebas

Ejecutar:

```
python manage.py test
```

Pruebas existentes:

```
tests/test_system.py
```

Cubren:

- creación con nombre y RUT;
- búsqueda;
- pagos y saldo;
- documentos en carpeta del cliente;
- normalización de RUT;
- descarga de PDF.

7.11 Agregar un Campo Nuevo

Ejemplo: agregar `numero_causa`.

Editar `clientes/models.py`:

```
numero_causa = models.CharField("Número de causa", max_length=80, blank=True)
```

Editar `clientes/templates/clientes/client_form.html`:

```
{% include 'clientes/_field.html' with field=form.numero_causa span='span-3' %}
```

Crear migración:

```
python manage.py makemigrations clientes
python manage.py migrate
```

7.12 Modificar el Contrato

Contrato HTML:

```
clientes/templates/clientes/contract_print.html
```

PDF alternativo:

```
clientes/pdf_fallback.py
```

Si cambia el diseño del contrato, revise ambos archivos.

7.13 Modificar Estilos

Archivo:

```
clientes/static/clientes/app.css
```

En producción, después de cambios:

```
sudo docker compose exec web python manage.py collectstatic --noinput
sudo docker compose restart nginx
```

O reconstruir:

```
sudo docker compose up -d --build
```

8. Seguridad Técnica

8.1 Variables Secretas

No publicar:

```
.env
db.sqlite3
media/
data/
respaldos reales
```

Sí publicar:

```
.env.example
código fuente
documentación
Dockerfile
docker-compose.yml
requirements.txt
```

8.2 Hosts Permitidos

En .env:

```
DJANGO_ALLOWED_HOSTS=localhost,127.0.0.1,192.168.0.22,servidor-funeraria.taild7534a.ts.net
```

8.3 CSRF por Tailscale

```
DJANGO_CSRF_TRUSTED_ORIGINS=https://servidor-funeraria.taild7534a.ts.net
```

8.4 No Exponer a Internet

No abrir el puerto 8080 del router hacia Internet.

Usar:

```
Tailscale Serve
```

Evitar:

```
Tailscale Funnel
```

9. Checklist de Administrador

Diario:

```
[ ] El sistema abre en Chrome.
[ ] Se puede iniciar sesión.
[ ] docker compose ps muestra servicios arriba.
```

Semanal:

```
[ ] Hay respaldos recientes.
[ ] El HDD tiene espacio.
[ ] Tailscale funciona.
[ ] No hay errores repetidos en logs.
```

Mensual:

```
[ ] Probar restauración en entorno separado.
[ ] Revisar usuarios activos.
[ ] Actualizar Ubuntu.
[ ] Copiar respaldos fuera del servidor.
```

10. Checklist de Programador

Antes de cambiar:

```
[ ] Hacer respaldo.
```

- [] Probar localmente.
- [] Ejecutar tests.
- [] Revisar migraciones.

Después de cambiar:

- [] Login funciona.
- [] Crear ficha funciona.
- [] Buscar por RUT funciona.
- [] Registrar pago funciona.
- [] Subir documento funciona.
- [] PDF funciona.
- [] Logs sin errores.

11. Comandos Rápidos

Estado:

```
cd /opt/sistema-funerario
sudo docker compose ps
```

Logs:

```
sudo docker compose logs --tail=100 web
```

Respaldar:

```
sudo ./deploy/scripts/backup.sh
```

Restaurar:

```
sudo ./deploy/scripts/restore.sh /mnt/funeraria-backups/AAAA-MM-DD_HH-MM-SS
```

Actualizar código:

```
sudo docker compose up -d --build
```

Crear usuario:

```
sudo docker compose exec web python manage.py createsuperuser
```

Cambiar contraseña:

```
sudo docker compose exec web python manage.py changepassword usuario
```

12. Conclusión

El administrador debe concentrarse en disponibilidad, usuarios, seguridad, respaldos y restauración.

El programador debe concentrarse en modelos, vistas, formularios, plantillas, pruebas y despliegues controlados.

La regla más importante para ambos perfiles:

Antes de tocar producción, hacer respaldo.